2018 4th IEEE International Conference on Big Data Security on Cloud

Method of Maintaining Data Consistency in Microservice architecture

Weibei Fan*§, Zhijie Han†§,Yujie Zhang‡, Ruchuan Wang§
*School of Computer Science and Technology
Soochow University, Suzhou, China
†Institute of Data and Knowledge Engineering
Henan University, Kaifeng, China
‡College of Computer
Nanjing University of Posts and Telecommunications, Nanjing, China
§Jiangsu High Technology Research Key Laboratory for Wireless Sensor Networks, Nanjing, China
Email: wbfan@stu.suda.edu.cn, hanzj@henu.edu.cn, zhangyujie@njupt.edu.cn, wangrc@njupt.edu.cn

***Abstract*—Data consistency is a classic problem in the field of distributed system research. The diversity and complexity of data sources and business needs in the big data era bring many new challenges to conformance research, and at the same time, push forward the research in this field. Microservice architecture is a new technology of deploying applications and services in the cloud. How to ensure data consistency in microservice has become a hot topic. We propose a method to improve the data consistency in microservice architecture.**

***Index Terms*—Big data; Data consistency; Distributed system; microservice.**

I. Introduction

In the era of big data, the data size, type and complexity increases with hitherto unknown speed [8], [11]. Meanwhile, the relationship between big data and the production life of the people becomes more closely, which makes distributed system as the basic platform to support big data under the current environment. Since data replication could improve the data persistence and availability, and also could make the system with strong fault tolerance and less latency in different regions. However, the replication mechanism has also brought some unavoidable costs, maintaining consistency between data copies will increase the burden of the system and affect the performance of the system [9], [14]. With the requirements of data center increase in recent years, the across geographic regions multi-data-centers is development rapidly [5], [7], [12]. It has become a copy to solve the scalability problem, the primary method of load balancing and high concurrency problems, which makes the consistency problem become the focus of research, which included data storage on the consistency and business processing.

Traditional single machine applications generally use a relational database, which has a advantage of ACID transactions using by applications [13]. To ensure consistency, we only need to start a transaction, change (insert, delete, update) many rows, and then commit transactions (if there are exceptions, rollback transactions). Furthermore, with the help of data access and framework (Spring) in the development platform, we do very little and only need to pay attention to the change of data itself.

As the scale of business organization continues to expand, stand-alone application and database has to support the business volume and the huge amount of data. It is time need to split the application and database, and to visit two or more than two of the database at the same time. We begin to use distributed transactions to ensure consistency, that is, the two-stage submission protocol (2PC) that we often say.

Microservice [1], [3] has gradually entered into the realization of everyone, and it is considered to be an effective means to realize the information of the next generation. To design the system, data consistency is not the key factor, but from local transaction to subsequent distributed transaction, it can effectively guarantee data consistency. But in the microservice architecture, these two ways are not the best choices.

II. The model of data consistency

Distributed systems improves the reliability and fault tolerance by copying data, and a copy of different data stored in different machines. The consistency of the cost to maintain a copy of the data is high. Some different consistency models have been proposed to improve performance for some systems with weak consistency.

- Strong consistency: it requires that no matter what copy of the update is executed, all reading operations will be able to get the latest data;
- Weak consistency: it takes some time for a user to read a certain operation for the update of the system, which we call the ”inconsistency window”;
- Final consistency: a special case of weak consistency that ensures the user can eventually read an operation to update the system specific data.

A. Quorum NRW system strategy

Replication technology, such as Quorum NRW system [10], which is the most effective technology to improve availability of SaaS. The protocol has three keywords, N, R, and W, where N represents the number of copies of the data, R represents the minimum number of copies that the read operation

978-1-5386-4399-0/18/$31.00 ©2018 IEEE
DOI 10.1109/BDS/HPSC/IDS18.2018.00023

IEEE computer society

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.

needs to read, ie, the minimum number of nodes to be involved in a read operation, W represents the minimum number of copies required to write a write operation, ie., the minimum number of nodes to be involved in a write operation.

In this strategy, the strong consistency can be guaranteed only by ensuring the $R + W > N$. For example, $N = 3$, $W = 2$, $R = 2$, which means that there are 3 different copies of data in the system. When writing operation, we need to wait for at least 2 copies finished. The write operation system will return to the successful execution state. For reading operation, the system has the same characteristics. Because of $R+W > N$, the system can guarantee strong consistency.

$R+W > N$ produces an effect similar to Quorum. The read (write) delay in this model is determined by the slowest copy of R (W). Sometimes, in order to get higher performance and smaller delay, the sum of R and W may be less than N. Bue the system cannot guarantee the reading operation while getting the latest data.

If $R + W > N$, the distributed system provides a strong consistency guarantee. Since nodes reading and writing data synchronously which is overlapped. In relational data management system, if N=2 can be set to W=2 and $R = 1$, this is a strong consistency constraint. The performance of write operation is relatively low, because the system needs data from 2 nodes to update, and then the confirmation result is returned to the user.

If $R = N+W$, the read and write operations are not overlap. System can only guarantee eventual consistency, and reached the same time copy implementation depends on the system of asynchronous update. Time inconsistency is also from the start to update, all nodes are asynchronous completion time between updates.

The setting of R and W directly affects the performance, scalability and consistency of the system. If W is set to 1, which is a complete copy of the change can be returned to the user. The copy through asynchronous mechanism to update the remaining $N - W$. if R is set to 1, there is a copy to read and complete the operation. The values of R and W are small and will affect the performance of consistency, thus these two values need to be weighed.

B. Two phase submission algorithm

In the two phase commit protocol [2], the system generally consists of two types of machines (or nodes). one is the coordinator (Coordinator), usually in a system with only one; Another one is transaction participants (participants, cohorts or workers), usually contains more than one. Data storage system can understand the number of copy data. The two phase submission agreement composed two stages. Under normal execution, the execution of the two stages is described as follows:

Phase 1: request phase (commit-request phase, or voting phase, voting phase). At the request stage, the coordinator will notify the transaction participant to submit, or cancel the transaction and then enter the voting process. During the voting process, participants will inform the coordinator of his decision: consent (transaction participant's local job execution is successful) or cancel (local job execution failure).

Phase 2: the submission phase (commit phase). At this stage, the coordinator will make decisions based on the results of the first stage of voting: submission or cancellation. If all participants agree to submit a transaction coordinator, they will notify all participants to submit the transaction. Otherwise, the coordinator will notify all participants to cancel the transaction. The participants will perform the response after receiving the messages sent by the coordinator. The two phase submission algorithm is combined in the distributed system, which can make the single user modify multiple copies of the file (object) and synchronize the multi copy data. The client receives all the response of the storage host, and the whole operation is finished.

III. Data consistency in distributed transactions

In the traditional click application, relational database is the simplest and most commonly method to ensure data consistency in relational database transactions. There are four basic elements of this transaction: ACID.

- A (Atomicity): all operations in the entire transaction are either completed, or all failed. It is impossible to stagnate in one of the links in the middle. An error occurs during the execution of a transaction, which is rolled back (Rollback) to the state before the start of the transaction. The transaction has never been executed;
- C (Consistency): a transaction can encapsulate a state change (unless it is a read-only). The transaction must always keep the system in a consistent state, regardless of the number of concurrent transactions at any time;
- I (Isolation): isolated state execution transactions. They seem to be the only operation that the system executes in a given time. If there are two transactions running in the same time and performing the same functions, the isolation of transactions will ensure every transaction considers that only the transaction is using in the system. This property is sometimes called serialization. In order to prevent confusion between transaction operations, serialization or serialization requests must be serialized. Only one request is applied to the same data at the same time;
- D (Durability, persistence): after transaction completion, the transaction's modification to the database is persisted in the database and will not be rolled back.

In traditional local transactions, in order to ensure data consistency, we only need to start a transaction first, then add, modify, delete and other operations, then submit transactions, if abnormal, roll back. Simply, it will be able to stand on the shoulders of the large database vendors to achieve data consistency.

As the scale of the organization is expanding and the volume of business is increasing, the single machine application is not enough to support the huge amount of business data. This needs to be disassembled for applications and data. There is a situation where multiple databases need to be accessed at the same time. At this time, distributed transactions are needed

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.

to ensure data consistency, that is, the commonly said 2PC, two Phase Commitment Protocol). In this protocol, the key point is that the activities of multiple databases are controlled by a component of a transaction coordinator. It is specifically divided into 5 steps:

- Application invokes the submission method in the transaction coordinator;
- The transaction coordinator will contact each database involved in the transaction and notify them to submit the transaction (this is the beginning of the first phase);
- After receiving the notification of preparing to submit a transaction, the database must ensure that it can commit transactions when asked to submit transactions or rollback transactions when required to rollback transactions. If the database is unable to prepare a transaction, it responds to the transaction coordinator with a negative response;
- The transaction coordinator collects all the responses from each database;
- In the second phase, the transaction coordinator notifies the result of the transaction to each database. If any database makes a negative response, the transaction coordinator sends a rollback command to all the databases involved in the transaction. If the database makes an affirmative response, the transaction coordinator will instruct all the resource managers to submit the transaction. Once the database is notified, the transaction will not fail. By responding to the first phase in an affirmative way, each resource manager has ensured that if it is later notified that it submits a transaction, the transaction will not fail.

In traditional system architecture, database is usually used as resource manager, and data consistency is guaranteed by transaction. Even in real distributed transactions, data consistency can be achieved by using database transactions.

In the microservice architecture, data access is complicated. Normally, data are private and can only be accessed by API. This approach can achieve loose coupling between micro services, making it easier to expand each other's independent micro services. But one problem is that it is not clear that the underlying data storage (not necessarily relational database) can not achieve data consistency through a unified transaction coordinator. In short, traditional local or distributed transactions are not suitable for the microservice architecture.

IV. Data consistency in microservice

Microservice first jointly proposed by Martin Fowler and James Lewis in 2014, which is a way to use a small service to develop a single application, each service runs in its own process, and the use of lightweight communication mechanism, usually HTTP API, the service ability construction based on, and be able to through the automatic deployment mechanism for independent deployment, these services use different programming language, and different data storage technology, and maintain a minimum of centralized management.

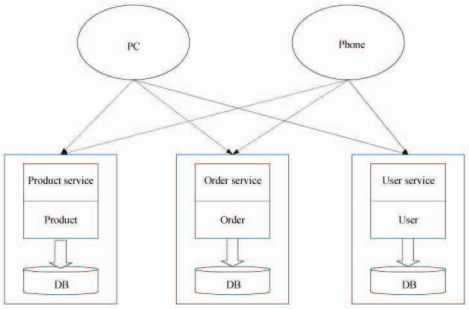


Fig. 1. Microservice architecture.

The single structure works well in small scale, but as the scale of the system expands, the problem is more and more exposed. The complexity is getting higher and higher. For example, there are hundreds of thousands of lines of code in some projects. The difference between modules is fuzzy. Logic is more chaotic. The more complexity of code, the harder it is to solve the problems. The speed of deployment gradually slows down. The single architecture module is very large, and the code is so large that it takes more and more time to deploy the project.

It is not only the key to micro services, but the system will provide a set of basic architecture, this architecture makes the micro services can be deployed independently, operation and upgrading, not only that, the system architecture also allows micro services and microservice between the structure of "loose coupling", while in function performance as a unified whole. The so-called "unified whole" shows the unified style interface, unified authority management, unified security policy, unified on-line process, unified log and audit method, unified scheduling mode, unified access entry and so on.

In distributed system architecture, there is a CAP Theory [6]: any distributed system can only satisfy three points of consistency, availability and partition tolerance at the same time. For distributed systems, partitioned fault tolerance is the basic requirement, otherwise it loses its value. Therefore, the choice between availability and consistency can only be made. If you choose to provide consistency, you need to pay for the cost of blocking other concurrent access before meeting consistency. This may continue an uncertain time, especially when the system has shown high delay or the network failure leads to a loss of connection. According to the current successful experience, usability is generally a better choice, but maintaining data consistency between services and databases is a fundamental requirement. Microservice architecture chooses to meet final consistency.

From the essence of consistency, all services that are included in a business logic are either successful or failed. So how do we choose the direction to ensure success or failure? That is, you need to make a choice according to the business model. There are three modes to achieve the final consistency: the reliable event pattern, the business compensation model, and the Try-Confirm-Cancel(TCC) model.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.

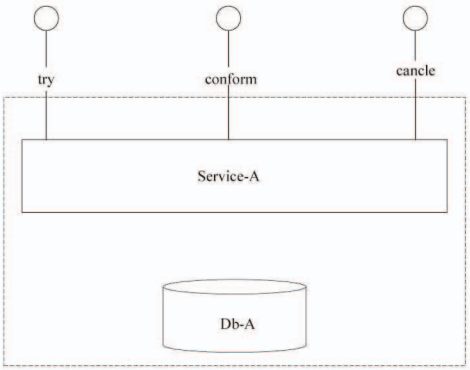


Fig. 2. Try-Confirm-Cancel Model.

(1) Reliable event mode: This model belongs to event driven architecture. When something important happens, for example, updating a business entity, microservice will publish an event to the message agent. Message broker can push events to subscribe to events. When receiving these events, they can finish their business, or they may cause more events.

(2) Business compensation model: Before describing the business compensation pattern, two concepts are defined first: Business anomaly: business logic is wrong, such as insufficient account balance, inadequate inventory of goods, and so on. Technical exception: the exception of non business logic, such as network connection exception, network timeout, and so on.

(3) Try-Confirm-Cancel: A complete TCC business is composed of a main business service and several business services. The main business service initiates and completes the entire business activity. The TCC mode requests to provide three interfaces from the service: Try, Confirm and Cancel. Try: complete all business checks and reserve necessary business resources. Confirm: the real execution of business, without any business inspection, only using the Try phase of the reserved business resources, Confirm operation to satisfy idempotent. Cancel: release the service resources reserved for the Try phase and the Cancel operation to satisfy idempotency.

The entire TCC service is divided into two stages:

The first phase: the primary service invokes all try operations from the business and registers all the service services in the activity manager. When all try operations from the service service are invoked successfully or a try operation from the service service fails, it enters the second stage.

The second phase: the activity manager performs confirm or cancel operations according to the execution results of the first phase. If all the try operations are successful in the first phase, the activity manager calls all the confirm operations from the business activity. Otherwise, all cancel operations from the service service are invoked.

V. Conclusion

In this paper, the definition of consistency problem in the domain of database and distributed domain is first given, and the research content of the consistency problem of distributed system is clearly defined. Then, from the perspective of microservice, this paper comprehensively introduces the existing consistency models, and makes in-depth analysis, so as to get the business scenarios applicable to different consistency models. Finally, this paper summarizes the research status of the existing consistency model, and points out the development of future consistency research in combination with the trend of technology development in the large data age. Due to the lack of indicators for unified evaluation of various consistency models, therefore, future design research consistency evaluation criteria [4] will be very meaningful.

Acknowledgements

The subject is sponsored by the National Natural Science Foundation of P. R. China (No.61572260, No.61572261, No.61672296, No.61672209, No.61602261, No.61701170), the Natural Science Foundation of Jiangsu Province (No.BK20140888, No.BK20160089), China Postdoctoral Science Foundation(2014M560439), Jiangsu Planned Projects for Postdoctoral Research Funds(1302084B), Scientific & Technological Support Project of Jiangsu Province (No.BE2015702, No.BE2016777, No.BE2016185), Jiangsu High Technology Research Key Laboratory for Wireless Sensor Networks Foundation (No.WSNLBKF201701).

References

[1] A. Balalaie, A. Heydarnoori, P. Jamshidi. Microservices Architecture Enables DevOps: an Experience Report on Migration to a Cloud-Native Architecture. IEEE Software, 2016, 33(3):42-52.

[2] M. Böhm, J. Sgall, R. V. Stee. A two-phase algorithm for bin stretching with stretching factor 1.5. Journal of Combinatorial Optimization, 2017:1-19.

[3] N. Dragoni, I. Lanese, S. T. Larsen. Microservices: How To Make Your Application Scale, In Proceedings of International Andrei Ershov Memorial Conference on Perspectives of System Informatics. Springer, Cham, 2017:95-104.

[4] W. B. Fan, Z, J. Han, R. Wang. An Evaluation Model and Benchmark for Parallel Computing Frameworks. Mobile Information Systems. 2018, In Press.

[5] J. Garcia-Dorado, S. Rao. Cost-aware Multi Data-Center Bulk Transfers in the Cloud from a Customer-Side Perspective. IEEE Transactions on Cloud Computing, 2015, (99):1-1.

[6] S. Gilbert, N. Lynch. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. ACM, 2002.

[7] P. Li, X. Yu, H. Xu H. Research on Secure Localization Model based on Trust Valuation in Wireless Sensor Networks. Security & Communication Networks, 2017(4) 1-12.

[8] Y. Li, X. Qi, M. Keally, Z. Ren, G. Zhou, D. Xiao, S. Deng, Communication energy modeling and optimization through joint packet size analysis of BSN and wifi networks, IEEE Transaction on Parallel and Distributed Systems. 24 (9) (2013) 1741-1751.

[9] P. Li, H. Nie, L. Qiu, R. Wang. Energy optimization of ant colony algorithm in wireless sensor network. International Journal of Distributed Sensor Networks, 2017, 13(4):155014771770483.

[10] L. Kong, Q. Li, L. Li L. Research on Multi-tenant Replication Consistency Based on Quorum NRW System. International Journal of Grid & Distributed Computing, 2014, 7.

[11] L. Kuang, L.T. Yang, Y. Liao, An integration framework on cloud for cyber physical social systems big data, IEEE Transaction on Cloud Computing. (2015). pp(9), 1-1.

[12] S. Ma, G. Si, W. Yue W. An online monitoring measure consistency computing algorithm by sliding window in multi-sensor system, In Proceedings of 2016 IEEE International Conference on Mechatronics and Automation (ICMA), 2016: 2185-2190.

[13] V. Narasayya, I. Menache, M. Singh. Sharing buffer pool memory in multi-tenant relational database-as-a-service. In Proceedings of the Vldb Endowment, 2015, 8(7):726-737.

[14] D. Terry. Replicated Data Consistency Explained Through Baseball. Communications of the Acm, 2013, 56(12):82-89.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.